

PRN: 24070521238

Experiment 1

Title: Demonstrate Inline, Internal, and External JavaScript; Use Console Methods; Create a Webpage Showing User Information and a Welcome Message

Software/Tools Required

- Visual Studio Code (VS Code) or any Text Editor
- Web Browser (Google Chrome, Microsoft Edge, Mozilla Firefox, etc.)
- HTML
- JavaScript

Code

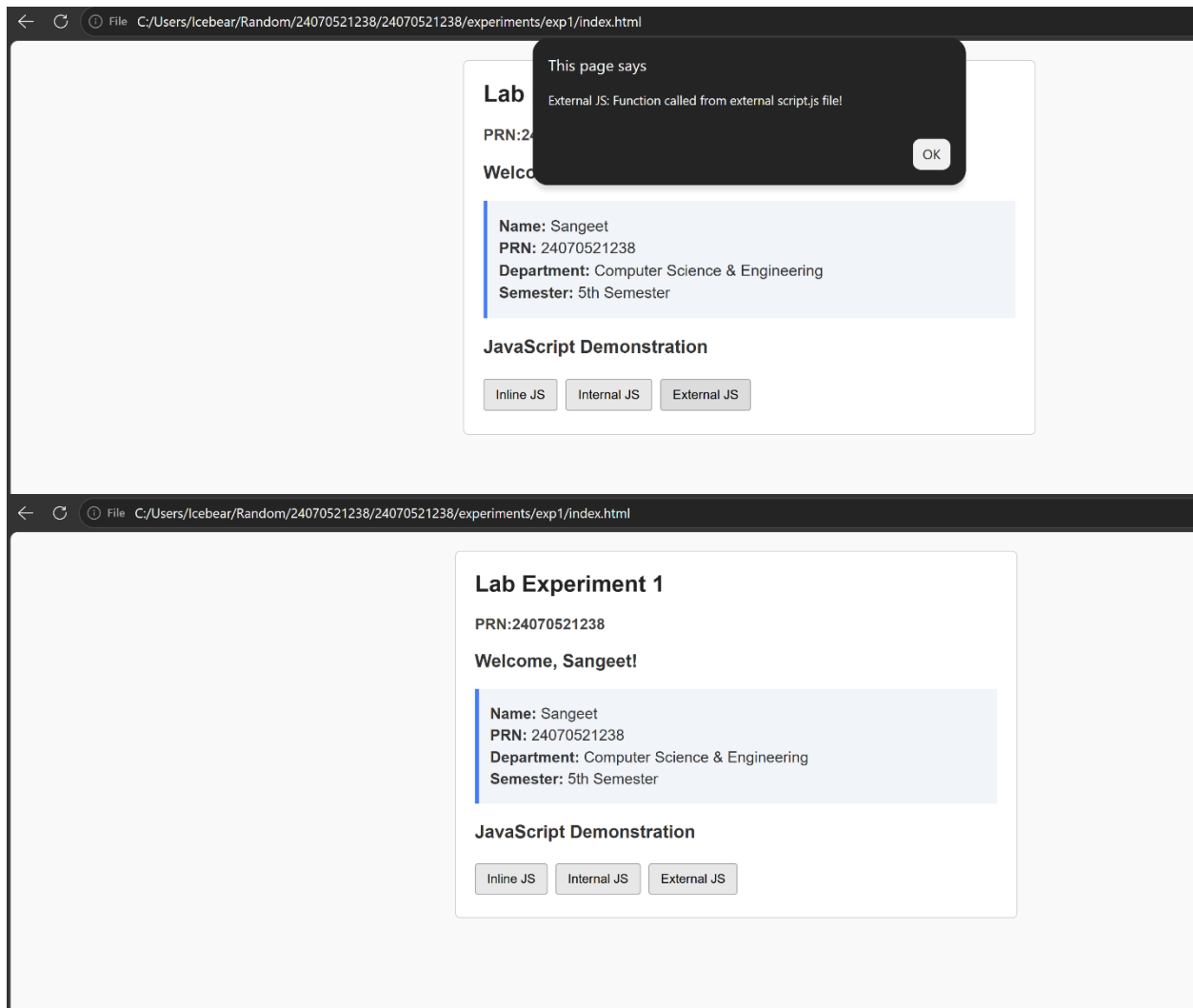
```
<!DOCTYPE html>
<html lang="en">
•
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Lab Experiment - JavaScript Demonstration</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 30px;
      background-color: #f4f4f4;
      color: #333;
    }
    •
    .card {
      background-color: #ffffff;
      border: 1px solid #ddd;
      padding: 20px;
      max-width: 550px;
      border-radius: 5px;
    }
    •
    .user-info {
      background-color: #e9ecef;
      padding: 12px;
      border-left: 4px solid #007bff;
      margin: 15px 0;
    }
    •
    button {
      padding: 8px 12px;
      margin-right: 5px;
      margin-bottom: 5px;
      cursor: pointer;
    }
  </style>
  •
  <!-- Internal JavaScript -->
  <script>
    // Console methods in Internal JS
    console.log("Internal JS: Script loaded in head.");
    console.info("Internal JS Info: Page title is '" + document.title + "'");
    •
    function internalJsDemo() {
      console.log("Internal JS: Button clicked!");
      alert("Hello from Internal JavaScript!");
    }
  </script>
</head>
```

```

•
<body>
•
  <div class="card">
    <h1>JavaScript Basics Lab</h1>
    •
    <!-- Dynamic Welcome Message -->
    <h2 id="welcome-msg">Welcome!</h2>
    •
    <!-- User Information Section -->
    <div class="user-info">
      <h3>User Information</h3>
      <p><strong>Name:</strong> Sangeet Harne</p>
      <p><strong>Roll No:</strong> 24070521238</p>
      <p><strong>Branch:</strong> Computer Science & Engineering</p>
    </div>
    •
    <h3>Demonstrate JavaScript Execution Types</h3>
    •
    <!-- Inline JavaScript -->
    <button onclick="console.log('Inline JS triggered'); alert('Hello from Inline
JavaScript!');">
      Inline JS
    </button>
    •
    <!-- Internal JavaScript -->
    <button onclick="internalJsDemo()">
      Internal JS
    </button>
    •
    <!-- External JavaScript -->
    <button onclick="externalJsDemo()">
      External JS
    </button>
  </div>
  •
  <!-- External JavaScript File -->
  <script src="script.js"></script>
</body>
•
</html>

```

Output



Result / Conclusion

The experiment was performed successfully. Inline, internal, and external JavaScript were implemented in an HTML webpage. Different console methods were used to display messages for debugging purposes, and the webpage successfully displayed user information along with a personalized welcome message. This experiment demonstrated the different ways of integrating JavaScript into web pages and highlighted the use of console methods during development.

Case Study 1

Title: Create a webpage for the college department and link the Student Information page with it, and write a JavaScript program to display “Welcome to SIT Nagpur” in an alert box.

Code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Department Page - SIT Nagpur</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 20px;
      background-color: #f4f4f4;
    }
    .header {
      background-color: #003366;
      color: white;
      padding: 15px;
      text-align: center;
    }
    .prn {
      font-size: 16px;
      font-weight: bold;
      color: #333;
      margin: 10px 0;
    }
    .content {
      background: white;
      padding: 20px;
      margin-top: 15px;
      border: 1px solid #ccc;
    }
    a {
      color: #003366;
      font-weight: bold;
    }
  </style>
</head>
<body>

  <script>
    alert("Welcome to SIT Nagpur");
  </script>

  <div class="header">
    <h1>Department of Computer Science & Engineering</h1>
    <p>Symbiosis Institute of Technology, Nagpur</p>
  </div>

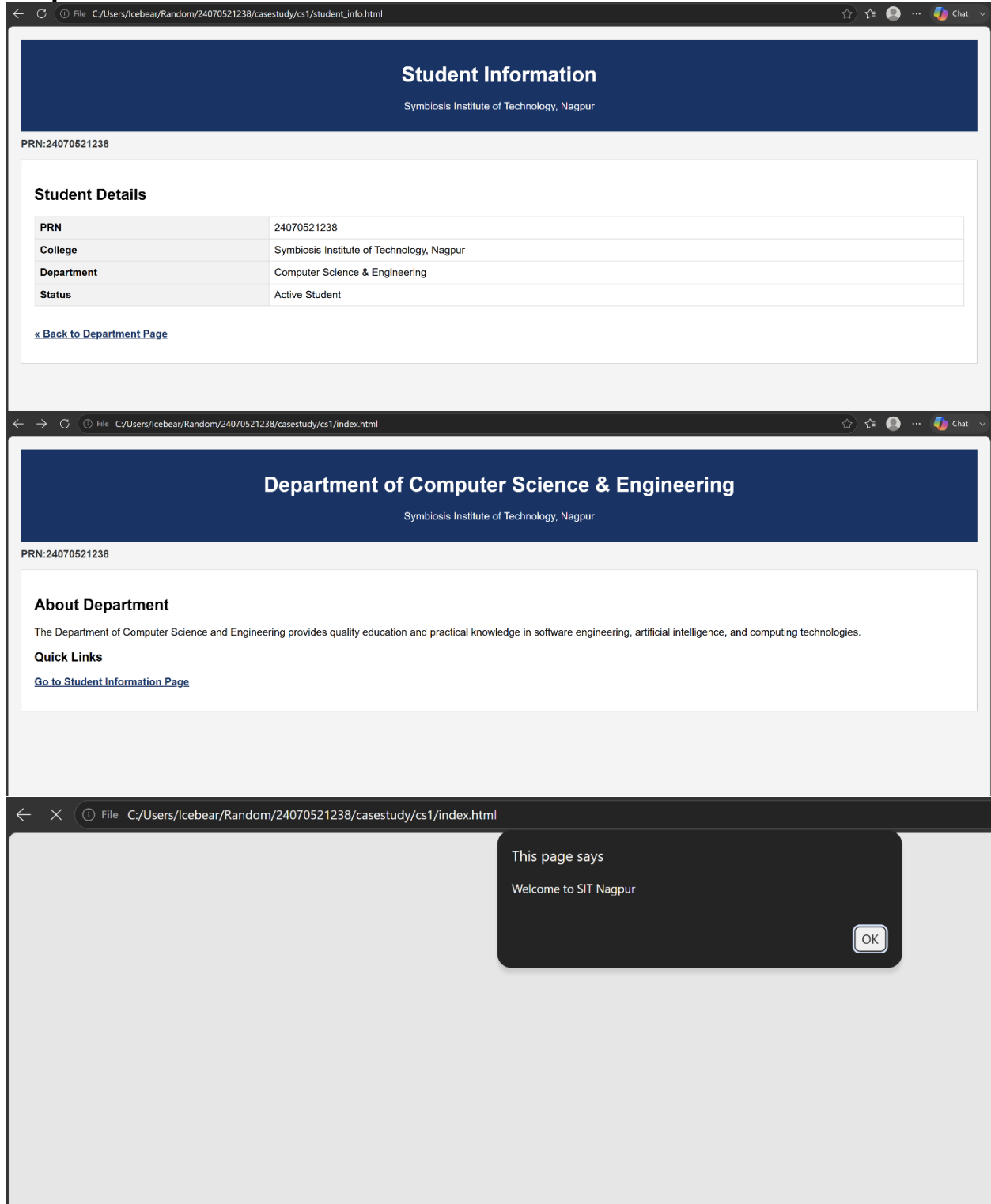
  <div class="prn">PRN:24070521238</div>

  <div class="content">
    <h2>About Department</h2>
    <p>The Department of Computer Science and Engineering provides quality education and practical knowledge in software engineering, artificial intelligence, and computing technologies.</p>

    <h3>Quick Links</h3>
    <p><a href="student_info.html">Go to Student Information Page</a></p>
  </div>
```

```
</body>
</html>
```

Output



Result / Conclusion

The college department webpage was successfully created using HTML. A link to the **Student Information** page was added and tested successfully. JavaScript was also implemented to display “**Welcome to SIT Nagpur**” in an alert box when the webpage is opened.

Experiment 2

Title: Use var, let, const, and Demonstrate Basic Features like Template Literals and Destructuring; Build a Billing Calculator with User Input

Software/Tools Required

- Visual Studio Code (VS Code) or any Text Editor
- Web Browser (Google Chrome, Microsoft Edge, Mozilla Firefox, etc.)
- HTML
- JavaScript

Code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Interactive Billing Calculator</title>
</head>
<body>
  <div class="box">
    <div class="prn">
      <table>
        <thead>
          <tr>
            <th>Item</th>
            <th>Price</th>
            <th>Quantity</th>
            <th>Total</th>
          </tr>
        </thead>
        <tbody>
          <tr>
            <td><input type="text" value="Item">
            <td><input type="text" value="Price">
            <td><input type="text" value="Quantity">
            <td><input type="text" value="Total">
          </tr>
        </tbody>
      </table>
    </div>
    <div class="btn">
      <button>Calculate</button>
    </div>
    <div class="result">
      <div>Result</div>
    </div>
  </div>
</body>
</html>
```



```

    ul {
      padding-left: 20px;
    }
  </style>
</head>
<body>

<div class="box">
  <h2>Interactive Billing Calculator</h2>
  <div class="prn">PRN:24070521238</div>

  <table>
    <thead>
      <tr>
        <th>Item Name</th>
        <th>Price (₹)</th>
        <th>Qty</th>
        <th>Action</th>
      </tr>
    </thead>
    <tbody id="itemList">
      <!-- Dynamic items will be added here -->
    </tbody>
  </table>

  <div style="margin-bottom: 15px;">
    <label>Tax Rate (%): </label>
    <input type="number" id="taxRate" value="10" style="width: 70px;">
  </div>

  <button class="btn" onclick="addItem()">+ Add Item</button>
  <button class="btn" onclick="calculateBill()">Calculate Total</button>

  <div id="output" class="result" style="display: none;"></div>
</div>

<script>
  // 1. Using 'var'
  var currencySymbol = "₹";

  // Function to add a new item row
  function addItem(itemName = "", itemPrice = "", itemQty = 1) {
    const itemList = document.getElementById('itemList');

    // 2. Template Literals for row creation
    const rowHTML = `
      <tr>
        <td><input type="text" class="item-name" value="${itemName}" placeholder="Item
name"></td>
        <td><input type="number" class="item-price" value="${itemPrice}" placeholder="0"></td>
        <td><input type="number" class="item-qty" value="${itemQty}" placeholder="1"></td>
        <td><button
class="btn"
onclick="this.parentElement.parentElement.remove()">Delete</button></td>
      </tr>
    `;
    itemList.insertAdjacentHTML('beforeend', rowHTML);
  }

  // Default sample items
  addItem("Notebook", 50, 2);
  addItem("Pen", 10, 5);

  function calculateBill() {
    // 3. Using 'const'
    const names = document.querySelectorAll('.item-name');
    const prices = document.querySelectorAll('.item-price');
    const qtys = document.querySelectorAll('.item-qty');
    const taxInput = parseFloat(document.getElementById('taxRate').value) || 0;

    // 4. Using 'let'
    let subtotal = 0;

```

```

let billItems = [];

for (let i = 0; i < names.length; i++) {
  const name = names[i].value || "Unnamed Item";
  const price = parseFloat(prices[i].value) || 0;
  const qty = parseInt(qtys[i].value) || 0;
  const itemTotal = price * qty;

  subtotal += itemTotal;

  billItems.push({ name, price, qty, itemTotal });
}

let taxAmount = (subtotal * taxInput) / 100;
let grandTotal = subtotal + taxAmount;

let itemsListHTML = "";

// 5. Object Destructuring & Template Literals
for (const itemObj of billItems) {
  const { name, qty, itemTotal } = itemObj;
  itemsListHTML += `<li>${name} (x${qty}) - ${currencySymbol}${itemTotal.toFixed(2)}</li>`;
}

const receiptHTML = `
  <strong>Bill Receipt</strong>
  <ul>${itemsListHTML}</ul>
  <p><strong>Subtotal:</strong> ${currencySymbol}${subtotal.toFixed(2)}</p>
  <p><strong>Tax (${taxInput}%):</strong> ${currencySymbol}${taxAmount.toFixed(2)}</p>
  <hr>
  <strong>Total Amount: ${currencySymbol}${grandTotal.toFixed(2)}</strong>
`;

const outputDiv = document.getElementById('output');
outputDiv.innerHTML = receiptHTML;
outputDiv.style.display = 'block';
}
</script>

</body>
</html>

```

Output

←

↻

File C:/Users/Icebear/Random/24070521238/experiments/exp2/index.html

Interactive Billing Calculator

PRN:24070521238

Item Name	Price (₹)	Qty	Action
Notebook	50	2	Delete
Pen	10	5	Delete

Tax Rate (%):

Bill Receipt

- Notebook (x2) - ₹100.00
- Pen (x5) - ₹50.00

Subtotal: ₹150.00

Tax (10%): ₹15.00

Total Amount: ₹165.00

Result / Conclusion

The experiment was performed successfully. The variables were declared using var, let, and const, and modern JavaScript features such as template literals and destructuring were implemented. A billing calculator was developed to accept user input, calculate the total bill including GST, and display the final amount. This experiment demonstrated the practical use of JavaScript variable declarations and ES6 features in creating interactive web applications.

Case Study 2

Title: Develop the webpage for an online shopping website using HTML,CSS and HTML5 tags And calculate the bill.

Code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Online Shopping & Bill Calculator</title>
  <style>
    * {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
      font-family: Arial, sans-serif;
    }

    body {
      background-color: #f4f6f9;
      color: #333;
      padding: 20px;
    }

    header {
      background-color: #2c3e50;
      color: white;
      padding: 15px 20px;
      display: flex;
      justify-content: space-between;
      align-items: center;
      border-radius: 6px;
      margin-bottom: 20px;
    }

    header h1 {
      font-size: 22px;
    }

    .prn-badge {
      font-size: 16px;
      font-weight: bold;
      background-color: #34495e;
      padding: 6px 12px;
      border-radius: 4px;
      color: #ecf0f1;
    }

    main {
      max-width: 800px;
      margin: 0 auto;
      display: grid;
      grid-template-columns: 1fr 1fr;
      gap: 20px;
    }

    @media (max-width: 600px) {
      main {
        grid-template-columns: 1fr;
      }
    }

    section {
      background: white;
    }
  </style>
</head>
```

```

        padding: 20px;
        border-radius: 6px;
        box-shadow: 0 2px 5px rgba(0,0,0,0.1);
    }

    h2 {
        font-size: 18px;
        margin-bottom: 15px;
        color: #2c3e50;
        border-bottom: 2px solid #ecf0f1;
        padding-bottom: 8px;
    }

    .product-item {
        display: flex;
        justify-content: space-between;
        align-items: center;
        margin-bottom: 12px;
        padding: 8px;
        border: 1px solid #e2e8f0;
        border-radius: 4px;
    }

    .product-info p {
        font-size: 14px;
        font-weight: bold;
    }

    .product-info span {
        font-size: 12px;
        color: #666;
    }

    .quantity-input {
        width: 60px;
        padding: 5px;
        text-align: center;
        border: 1px solid #ccc;
        border-radius: 4px;
    }

    button {
        width: 100%;
        padding: 10px;
        background-color: #27ae60;
        color: white;
        border: none;
        border-radius: 4px;
        font-size: 16px;
        cursor: pointer;
        margin-top: 15px;
    }

    button:hover {
        background-color: #219150;
    }

    .bill-summary {
        margin-top: 10px;
    }

    .bill-row {
        display: flex;
        justify-content: space-between;
        margin-bottom: 8px;
        font-size: 14px;
    }

    .bill-total {
        display: flex;
        justify-content: space-between;
    }

```

```

        margin-top: 12px;
        padding-top: 10px;
        border-top: 2px dashed #ccc;
        font-weight: bold;
        font-size: 16px;
        color: #2c3e50;
    }

    footer {
        text-align: center;
        margin-top: 25px;
        font-size: 13px;
        color: #777;
    }
}
</style>
</head>
<body>

    <!-- Header Section -->
    <header>
        <h1>GadgetHub Store</h1>
        <div class="prn-badge">PRN:24070521238</div>
    </header>

    <!-- Main Content -->
    <main>
        <!-- Product Selection Section -->
        <section>
            <h2>Select Products</h2>
            <form id="shopForm">
                <article class="product-item">
                    <div class="product-info">
                        <p>Wireless Headphones</p>
                        <span>Price: &#8377;1,500</span>
                    </div>
                    <input type="number" id="p1" class="quantity-input" min="0" value="0">
                </article>

                <article class="product-item">
                    <div class="product-info">
                        <p>Smart Watch</p>
                        <span>Price: &#8377;2,500</span>
                    </div>
                    <input type="number" id="p2" class="quantity-input" min="0" value="0">
                </article>

                <article class="product-item">
                    <div class="product-info">
                        <p>Gaming Mouse</p>
                        <span>Price: &#8377;800</span>
                    </div>
                    <input type="number" id="p3" class="quantity-input" min="0" value="0">
                </article>

                <article class="product-item">
                    <div class="product-info">
                        <p>Mechanical Keyboard</p>
                        <span>Price: &#8377;3,000</span>
                    </div>
                    <input type="number" id="p4" class="quantity-input" min="0" value="0">
                </article>

                <button type="button" onclick="calculateBill()">Calculate Total Bill</button>
            </form>
        </section>

        <!-- Bill Calculation Summary -->
        <section>
            <h2>Order Summary</h2>
            <div class="bill-summary" id="billResult">

```

```

        <p style="color: #888; font-size: 14px;">Select items and click calculate to view
bill.</p>
    </div>
</section>
</main>

<!-- Footer -->
<footer>
    <p>&copy; 2026 Online Shopping Website Assignment</p>
</footer>

<!-- JavaScript logic -->
<script>
    function calculateBill() {
        // Prices of products
        const price1 = 1500;
        const price2 = 2500;
        const price3 = 800;
        const price4 = 3000;

        // Quantities
        const q1 = parseInt(document.getElementById('p1').value) || 0;
        const q2 = parseInt(document.getElementById('p2').value) || 0;
        const q3 = parseInt(document.getElementById('p3').value) || 0;
        const q4 = parseInt(document.getElementById('p4').value) || 0;

        const subtotal = (q1 * price1) + (q2 * price2) + (q3 * price3) + (q4 * price4);

        if (subtotal === 0) {
            document.getElementById('billResult').innerHTML = '<p style="color: red; font-size:
14px;">Please enter quantity for at least one item.</p>';
            return;
        }

        // GST Tax @ 18%
        const gst = subtotal * 0.18;
        const totalBill = subtotal + gst;

        let summaryHTML = '';
        if (q1 > 0) summaryHTML += '<div class="bill-row"><span>Wireless Headphones
(${q1}):</span> <span>&#8377;${q1 * price1}</span></div>`;
        if (q2 > 0) summaryHTML += '<div class="bill-row"><span>Smart Watch (${q2}):</span>
<span>&#8377;${q2 * price2}</span></div>`;
        if (q3 > 0) summaryHTML += '<div class="bill-row"><span>Gaming Mouse (${q3}):</span>
<span>&#8377;${q3 * price3}</span></div>`;
        if (q4 > 0) summaryHTML += '<div class="bill-row"><span>Mechanical Keyboard
(${q4}):</span> <span>&#8377;${q4 * price4}</span></div>`;

        summaryHTML += '<hr style="margin: 8px 0; border: none; border-top: 1px solid #eee;">';
        summaryHTML += '<div class="bill-row"><span>Subtotal:</span>
<span>&#8377;${subtotal.toFixed(2)}</span></div>';
        summaryHTML += '<div class="bill-row"><span>GST (18%):</span>
<span>&#8377;${gst.toFixed(2)}</span></div>';
        summaryHTML += '<div class="bill-total"><span>Total Bill:</span>
<span>&#8377;${totalBill.toFixed(2)}</span></div>';

        document.getElementById('billResult').innerHTML = summaryHTML;
    }
</script>
</body>
</html>

```

Output

GadgetHub Store

PRN:24070521238

Select Products

Wireless Headphones

Price: ₹1,500

4

Smart Watch

Price: ₹2,500

4

Gaming Mouse

Price: ₹800

2

Mechanical Keyboard

Price: ₹3,000

1

Calculate Total Bill

Order Summary

Wireless Headphones (4):

₹6000

Smart Watch (4):

₹10000

Gaming Mouse (2):

₹1600

Mechanical Keyboard (1):

₹3000

Subtotal:

₹20600.00

GST (18%):

₹3708.00

Total Bill:

₹24308.00

Result / Conclusion

The online shopping webpage was successfully developed using **HTML, CSS, and HTML5 tags**. The webpage displays the available products along with their details and provides a user-friendly interface for online shopping. The total bill amount was successfully calculated based on the selected products and their quantities.

Experiment 3

Title: Implement Control Structures and Form Validation; Create a Grading System Based on User-Entered Marks

Software/Tools Required

- Visual Studio Code (VS Code) or any Text Editor
- Web Browser (Google Chrome, Microsoft Edge, Mozilla Firefox, etc.)
- HTML
- JavaScript

Code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Grading System</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 20px;
      background-color: #f4f4f4;
    }
    .container {
      max-width: 550px;
      margin: 20px auto;
      background: #fff;
      padding: 20px;
      border: 1px solid #ccc;
      border-radius: 6px;
    }
    h2 {
      text-align: center;
      margin-bottom: 5px;
    }
    .prn {
      text-align: center;
      font-size: 16px;
      font-weight: bold;
      color: #333;
      margin-bottom: 20px;
    }
    .form-group {
      margin-bottom: 12px;
    }
    label {
      display: inline-block;
      width: 120px;
    }
    input {
      padding: 6px;
      width: 200px;
    }
    button {
      padding: 6px 14px;
      cursor: pointer;
      margin-top: 5px;
    }
    .error {
      color: red;
      font-size: 14px;
    }
  </style>
</head>
<body>
  <h2>Student Grading System</h2>
  <div class="prn">
    <div class="form-group">
      <div class="label">Name</div>
      <input type="text">
    </div>
    <div class="form-group">
      <div class="label">Roll No</div>
      <input type="text">
    </div>
    <div class="form-group">
      <div class="label">Marks</div>
      <input type="text">
    </div>
    <button>Submit</button>
  </div>
  <div class="error">
    <div class="label">Error Message</div>
    <input type="text">
  </div>
</body>
</html>
```

```

        margin-top: 8px;
    }
    table {
        width: 100%;
        border-collapse: collapse;
        margin-top: 15px;
    }
    table, th, td {
        border: 1px solid #999;
        padding: 8px;
        text-align: center;
    }
    th {
        background-color: #e0e0e0;
    }
    .summary {
        margin-top: 15px;
        padding: 10px;
        background: #eef;
        border-radius: 4px;
    }
}
</style>
</head>
<body>

<div class="container">
    <h2>Student Grading System</h2>
    <div class="prn">PRN: 24070521238</div>

    <form id="gradeForm" onsubmit="return addSubject(event)">
        <div class="form-group">
            <label for="subject">Subject Name:</label>
            <input type="text" id="subject" placeholder="Enter subject name">
        </div>
        <div class="form-group">
            <label for="marks">Marks (0-100):</label>
            <input type="number" id="marks" placeholder="Enter marks">
        </div>
        <button type="submit">Add Subject</button>
        <div id="errorMsg" class="error"></div>
    </form>

    <h3>Subject Results</h3>
    <table>
        <thead>
            <tr>
                <th>Sr No.</th>
                <th>Subject</th>
                <th>Marks</th>
                <th>Grade</th>
                <th>Status</th>
            </tr>
        </thead>
        <tbody id="subjectTable">
            <!-- Dynamic rows will be inserted here -->
        </tbody>
    </table>

    <div class="summary" id="summaryBox" style="display: none;">
        <p><strong>Total Subjects:</strong> <span id="totalSubjects">0</span></p>
        <p><strong>Total Marks:</strong> <span id="totalMarks">0</span></p>
        <p><strong>Percentage:</strong> <span id="percentage">0</span>%</p>
        <p><strong>Overall Grade:</strong> <span id="overallGrade">-</span></p>
        <p><strong>Final Result:</strong> <span id="finalResult">-</span></p>
    </div>
</div>

<script>
    let subjectsList = [];

    // Function to calculate grade using control structures (if-else)

```

```

function getGrade(marks) {
  if (marks >= 90) return 'A+';
  else if (marks >= 80) return 'A';
  else if (marks >= 70) return 'B';
  else if (marks >= 60) return 'C';
  else if (marks >= 50) return 'D';
  else if (marks >= 40) return 'E';
  else return 'F';
}

// Form Validation and Add Subject
function addSubject(event) {
  event.preventDefault();

  const subjectInput = document.getElementById('subject');
  const marksInput = document.getElementById('marks');
  const errorMsg = document.getElementById('errorMsg');

  const subjectName = subjectInput.value.trim();
  const marksVal = marksInput.value.trim();
  const marks = parseFloat(marksVal);

  // Control structure: Form Validation
  if (subjectName === '') {
    errorMsg.innerText = 'Please enter a valid subject name.';
    return false;
  }

  if (marksVal === '' || isNaN(marks) || marks < 0 || marks > 100) {
    errorMsg.innerText = 'Marks must be a valid number between 0 and 100.';
    return false;
  }

  // Clear error message
  errorMsg.innerText = '';

  // Calculate Grade and Status
  const grade = getGrade(marks);
  const status = marks >= 40 ? 'Pass' : 'Fail';

  // Add subject object to array
  subjectsList.push({
    name: subjectName,
    marks: marks,
    grade: grade,
    status: status
  });

  // Clear input fields
  subjectInput.value = '';
  marksInput.value = '';

  // Update the display table and summary
  updateDisplay();
}

// Control structure: Loop to compute results and render table
function updateDisplay() {
  const tableBody = document.getElementById('subjectTable');
  tableBody.innerHTML = '';

  let totalMarks = 0;
  let hasFailed = false;

  // Loop through subjects list
  for (let i = 0; i < subjectsList.length; i++) {
    const sub = subjectsList[i];
    totalMarks += sub.marks;

    if (sub.status === 'Fail') {
      hasFailed = true;
    }
  }

```

```

    }

    const row = `<tr>
        <td>${i + 1}</td>
        <td>${sub.name}</td>
        <td>${sub.marks}</td>
        <td>${sub.grade}</td>
        <td style="color: ${sub.status === 'Pass' ? 'green' : 'red'}; font-weight:
bold;">${sub.status}</td>
    </tr>`;

    tableBody.innerHTML += row;
}

// Display summary section
const summaryBox = document.getElementById('summaryBox');
if (subjectsList.length > 0) {
    summaryBox.style.display = 'block';

    const count = subjectsList.length;
    const percentage = totalMarks / count;
    const overallGrade = getGrade(percentage);

    document.getElementById('totalSubjects').innerText = count;
    document.getElementById('totalMarks').innerText = totalMarks + ' / ' + (count * 100);
    document.getElementById('percentage').innerText = percentage.toFixed(2);
    document.getElementById('overallGrade').innerText = overallGrade;

    const finalResultElem = document.getElementById('finalResult');
    if (hasFailed) {
        finalResultElem.innerText = 'FAIL';
        finalResultElem.style.color = 'red';
    } else {
        finalResultElem.innerText = 'PASS';
        finalResultElem.style.color = 'green';
    }
}
}
</script>

</body>
</html>

```

Output

←

🔄

📄

File

C:/Users/Icebear/Random/24070521238/experiments/exp3/index.html

Student Grading System

PRN: 24070521238

Subject Name:

Enter subject name

Marks (0-100):

Enter marks

Add Subject

Subject Results

Sr No.	Subject	Marks	Grade	Status
1	Javascript	40	E	Pass
2	maths	30	F	Fail
3	chemistry	90	A+	Pass
4	physics	100	A+	Pass

Total Subjects: 4

Total Marks: 260 / 400

Percentage: 65.00%

Overall Grade: C

Final Result: FAIL

Result / Conclusion

The experiment was performed successfully. Control structures (if, if...else, and if...else if...else) were used to determine grades based on user-entered marks. Form validation ensured that only valid marks within the range of **0 to 100** were accepted. The program correctly displayed the corresponding grade and prevented invalid input, demonstrating the practical use of decision-making statements and input validation in JavaScript.

Case Study 3

Title: Write a JavaScript program that accepts student marks and displays the scholarship category.

Code

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Scholarship Category Checker</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 40px;
      background-color: #f9f9f9;
    }

    .box {
      width: 380px;
      background: #fff;
      padding: 20px;
      border: 1px solid #ccc;
      border-radius: 5px;
    }

    .prn {
      font-size: 16px;
      font-weight: bold;
      color: #333;
      margin-bottom: 15px;
    }

    label {
      font-size: 14px;
    }

    input {
      width: 90%;
      padding: 8px;
      margin-top: 4px;
      margin-bottom: 12px;
    }

    button {
      padding: 8px 15px;
      background-color: #28a745;
      color: white;
      border: none;
      cursor: pointer;
      width: 95%;
    }

    #result {
      margin-top: 15px;
      line-height: 1.5;
    }
  </style>
</head>

<body>

  <div class="box">
    <h2>Scholarship Checker</h2>
```

```

<div class="prn">PRN:24070521238</div>

<label for="sub1">Subject 1 Marks:</label><br>
<input type="number" id="sub1" min="0" max="100" placeholder="Marks for Subject 1"><br>

<label for="sub2">Subject 2 Marks:</label><br>
<input type="number" id="sub2" min="0" max="100" placeholder="Marks for Subject 2"><br>

<label for="sub3">Subject 3 Marks:</label><br>
<input type="number" id="sub3" min="0" max="100" placeholder="Marks for Subject 3"><br>

<label for="sub4">Subject 4 Marks:</label><br>
<input type="number" id="sub4" min="0" max="100" placeholder="Marks for Subject 4"><br>

<button onclick="checkScholarship()">Calculate Scholarship</button>

<div id="result"></div>
</div>

<script>
function checkScholarship() {
    let m1 = parseFloat(document.getElementById("sub1").value);
    let m2 = parseFloat(document.getElementById("sub2").value);
    let m3 = parseFloat(document.getElementById("sub3").value);
    let m4 = parseFloat(document.getElementById("sub4").value);

    let resultDiv = document.getElementById("result");

    if (isNaN(m1) || isNaN(m2) || isNaN(m3) || isNaN(m4) ||
        m1 < 0 || m1 > 100 || m2 < 0 || m2 > 100 ||
        m3 < 0 || m3 > 100 || m4 < 0 || m4 > 100) {
        resultDiv.innerHTML = "<span style='color:red;'>Please enter valid marks (0-100)
for all 4 subjects.</span>";
        return;
    }

    let total = m1 + m2 + m3 + m4;
    let avg = total / 4;
    let category = "";

    if (avg < 50) {
        category = "Not Eligible";
    } else if (avg >= 50 && avg <= 69) {
        category = "Bronze Scholarship";
    } else if (avg >= 70 && avg <= 84) {
        category = "Silver Scholarship";
    } else if (avg >= 85 && avg <= 94) {
        category = "Gold Scholarship";
    } else if (avg >= 95 && avg <= 100) {
        category = "Platinum Scholarship";
    }

    resultDiv.innerHTML = "<b>Average Marks:</b> " + avg.toFixed(2) + "%<br>" +
        "<b>Category:</b> " + category;
}
</script>
</body>
</html>

```

Output

← ↻ File C:/Users/Icebear/Random/24070521238/casestudy/cs3/index.html

Scholarship Checker

PRN:24070521238

Subject 1 Marks:

Subject 2 Marks:

Subject 3 Marks:

Subject 4 Marks:

Calculate Scholarship

Average Marks: 65.00%

Category: Bronze Scholarship

Result / Conclusion

The JavaScript program was successfully developed to accept the student's marks and display the appropriate scholarship category based on the marks obtained. The program was tested with different marks and produced the expected scholarship category successfully.

Experiment 4

Title: Use Function Types, Scope, and Closures; Apply Try-Catch; Build a Palindrome Checker

Software/Tools Required

- Visual Studio Code (VS Code) or any Text Editor
- Web Browser (Google Chrome, Microsoft Edge, Mozilla Firefox, etc.)
- HTML
- JavaScript

Code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Palindrome Checker</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 40px;
      background-color: #f4f4f4;
    }
    .container {
      max-width: 450px;
      background: #ffffff;
      padding: 20px;
      border: 1px solid #ccc;
      border-radius: 5px;
    }
    h2 {
      margin-top: 0;
    }
    .prn {
      font-size: 16px;
      font-weight: bold;
      color: #333333;
      margin-bottom: 15px;
    }
    input[type="text"] {
      width: 95%;
      padding: 8px;
      margin-bottom: 10px;
      border: 1px solid #ccc;
      border-radius: 3px;
    }
    button {
      padding: 8px 15px;
      cursor: pointer;
      background-color: #007bff;
      color: white;
      border: none;
      border-radius: 3px;
    }
    button:hover {
      background-color: #0056b3;
    }
    #result {
      margin-top: 15px;
      font-size: 16px;
      font-weight: bold;
    }
  </style>
</head>
<body>
  <h2>Palindrome Checker</h2>
  <div class="prn"></div>
  <input type="text">
  <button>Check</button>
  <div id="result"></div>
</body>
</html>
```

```

    }
    .error {
        color: red;
    }
    .success {
        color: green;
    }
    .tracker {
        margin-top: 10px;
        font-size: 14px;
        color: #555;
    }
}
</style>
</head>
<body>

<div class="container">
    <h2>Palindrome Checker</h2>
    <div class="prn">PRN: 24070521238</div>

    <label for="strInput">Enter Text:</label><br><br>
    <input type="text" id="strInput" placeholder="e.g., racecar or madam">
    <button onclick="handleCheck()">Check</button>

    <div id="result"></div>
    <div id="tracker" class="tracker"></div>
</div>

<script>
    // Global Scope Variable
    let globalCounter = 0;

    // Closure: Function returning inner function that retains outer scope variable
    function createHistoryTracker() {
        let checkCount = 0; // Outer variable preserved in closure
        return function() {
            checkCount++;
            return "Total Checks: " + checkCount;
        };
    }
    const trackCheck = createHistoryTracker();

    // Function Type 1: Function Expression (Anonymous Function)
    const cleanInput = function(text) {
        // Function Scope variable
        let cleaned = text.toLowerCase().replace(/[^a-z0-9]/g, '');
        return cleaned;
    };

    // Function Type 2: Arrow Function
    const isPalindrome = (str) => {
        let reversedStr = str.split('').reverse().join('');
        return str === reversedStr;
    };

    // Function Type 3: Function Declaration
    function handleCheck() {
        let inputElement = document.getElementById("strInput");
        let resultDiv = document.getElementById("result");
        let trackerDiv = document.getElementById("tracker");
        let inputValue = inputElement.value;

        // Try-Catch Block for Error Handling
        try {
            // Validation: Empty check
            if (!inputValue.trim()) {
                throw new Error("Please enter a string to check.");
            }

            let formattedText = cleanInput(inputValue);

```

```
    if (formattedText.length === 0) {
      throw new Error("Input must contain valid letters or numbers.");
    }

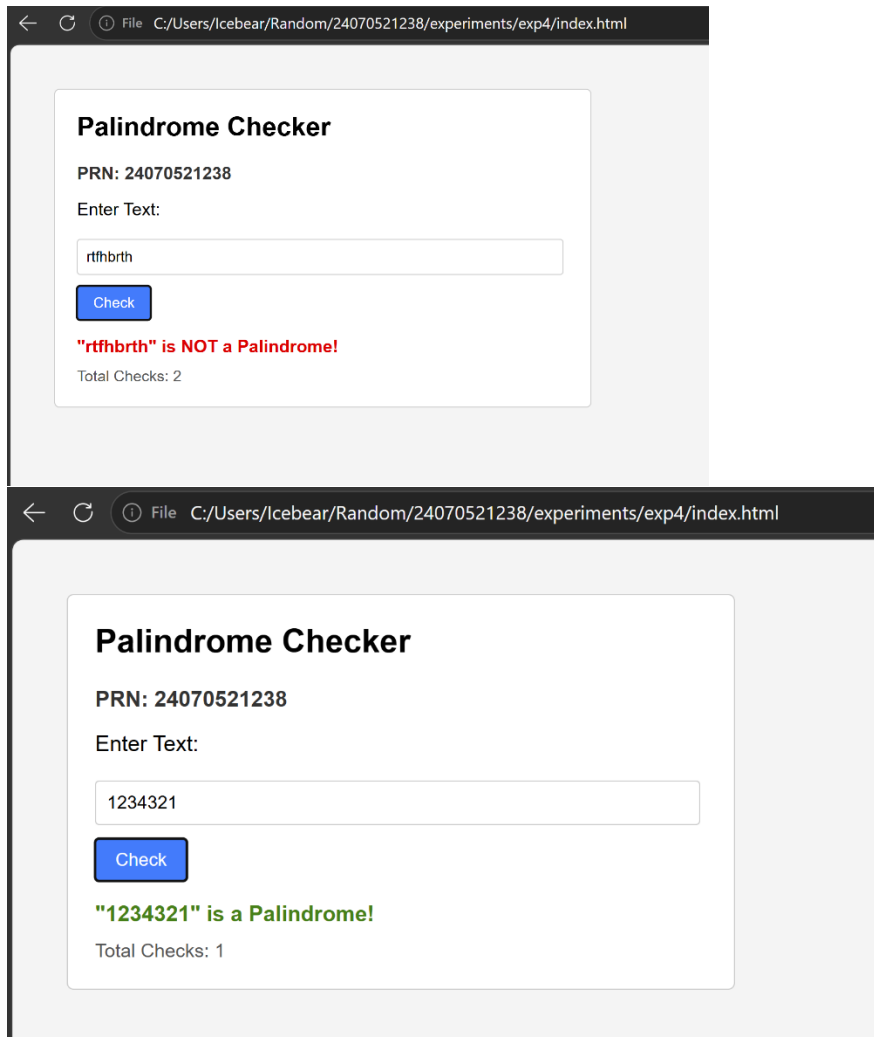
    // Check if string is a palindrome
    let result = isPalindrome(formattedText);

    if (result) {
      resultDiv.className = "success";
      resultDiv.innerHTML = `"${inputValue}" is a Palindrome!`;
    } else {
      resultDiv.className = "error";
      resultDiv.innerHTML = `"${inputValue}" is NOT a Palindrome!`;
    }

    // Update closure counter
    trackerDiv.innerText = trackCheck();
  } catch (err) {
    // Display catch error message
    resultDiv.className = "error";
    resultDiv.innerHTML = "Error: " + err.message;
  }
}
</script>

</body>
</html>
```

Output



Result / Conclusion

The experiment was performed successfully. Different JavaScript function types, including function declarations, function expressions, and arrow functions, were implemented. Variable scope and closures were demonstrated to show how data is accessed and preserved within functions. A palindrome checker was developed that accepted user input, determined whether the entered text was a palindrome, and displayed the appropriate result.

Case Study 4

Title: ATM Card PIN Verification Using JavaScript Functions : An ATM system verifies whether a customer's PIN is a palindrome. If the PIN reads the same forward and backward, the bank displays a special security message. To implement this, create a function to reverse the entered PIN number and another function to check whether it is a palindrome. Use function declaration, function expression, arrow function, and demonstrate scope and closure concepts.

Code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>ATM Card PIN Verification</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f4f6f9;
      margin: 0;
      padding: 40px 20px;
      display: flex;
      justify-content: center;
    }
    .container {
      background-color: #ffffff;
      padding: 25px 30px;
      border-radius: 8px;
      box-shadow: 0 4px 10px rgba(0,0,0,0.1);
      width: 100%;
      max-width: 440px;
      border: 1px solid #ddd;
    }
    h2 {
      margin-top: 0;
      color: #1a365d;
      font-size: 22px;
      text-align: center;
    }
    .prn-text {
      font-size: 16px;
      font-weight: 600;
      color: #4a5568;
      text-align: center;
      margin-bottom: 20px;
    }
    .form-group {
      margin-bottom: 15px;
    }
    label {
      display: block;
      margin-bottom: 5px;
      font-weight: bold;
      color: #333;
    }
    input[type="text"] {
      width: 100%;
      padding: 10px;
      box-sizing: border-box;
      border: 1px solid #ccc;
      border-radius: 4px;
      font-size: 16px;
    }
  </style>
</head>
<body>
  <div class="container">
    <h2>ATM Card PIN Verification</h2>
    <p>Enter your PIN:</p>
    <input type="text"/>
    <button type="button">Verify</button>
  </div>
</body>
</html>
```

```

        letter-spacing: 2px;
    }
    button {
        width: 100%;
        padding: 10px;
        background-color: #2b6cb0;
        color: white;
        border: none;
        border-radius: 4px;
        font-size: 16px;
        cursor: pointer;
        font-weight: bold;
    }
    button:hover {
        background-color: #2c5282;
    }
    .result {
        margin-top: 20px;
        padding: 15px;
        border-radius: 4px;
        display: none;
        line-height: 1.6;
    }
    .success {
        background-color: #e6fffa;
        border: 1px solid #b2f5ea;
        color: #234e52;
    }
    .normal {
        background-color: #ebf8ff;
        border: 1px solid #bee3f8;
        color: #2b6cb0;
    }
    .error {
        background-color: #fff5f5;
        border: 1px solid #fed7d7;
        color: #c53030;
    }
}
</style>
</head>
<body>

<div class="container">
    <h2>ATM PIN Verification System</h2>
    <div class="prn-text">PRN:24070521238</div>

    <form onsubmit="event.preventDefault(); verifyPIN();">
        <div class="form-group">
            <label for="pinInput">Enter ATM PIN:</label>
            <input type="text" id="pinInput" placeholder="e.g. 1221 or 4589" maxlength="6"
autocomplete="off">
        </div>

        <button type="submit">Verify PIN</button>
    </form>

    <div id="result" class="result"></div>
</div>

<script>
    // =====
    // DEMONSTRATION OF JAVASCRIPT CONCEPTS (COLLEGE ASSIGNMENT)
    // =====

    // 1. GLOBAL SCOPE VARIABLE
    // Accessible everywhere in the script
    const BANK_NAME = "Secure ATM Network";

    // 2. CLOSURE CONCEPT
    // Outer function returns inner function that retains access to 'attempts'
    function createAttemptTracker() {

```

```

    let attempts = 0; // Lexical environment variable (Closure state)
    return function() {
        attempts++;
        return attempts;
    };
}
// Instantiate closure
const getAttemptCount = createAttemptTracker();

// 3. FUNCTION DECLARATION
// Standard function declaration to reverse entered PIN
function reversePIN(pin) {
    let strPin = String(pin);
    let reversed = ""; // Local Function Scope variable
    for (let i = strPin.length - 1; i >= 0; i--) {
        reversed += strPin[i];
    }
    return reversed;
}

// 4. FUNCTION EXPRESSION
// Function expression stored in a variable to check palindrome
const checkPalindrome = function(pin) {
    let strPin = String(pin);
    let reversedPin = reversePIN(strPin); // Calls function declaration
    return strPin === reversedPin;
};

// 5. ARROW FUNCTION
// Arrow function to generate security message based on palindrome status
const getSecurityMessage = (isPalin) => {
    if (isPalin) {
        return "PIN ACCEPTED: Palindromic PIN Verified! Special security message activated.";
    } else {
        return "WRONG PIN: Access Denied! Entered PIN is not a palindrome.";
    }
};

// Main verification function
function verifyPIN() {
    // FUNCTION SCOPE VARIABLES
    let pin = document.getElementById("pinInput").value.trim();
    let resultDiv = document.getElementById("result");

    // Strict numeric input validation (digits only)
    if (pin === "" || !/^\d+$/.test(pin)) {
        resultDiv.className = "result error";
        resultDiv.style.display = "block";
        resultDiv.innerHTML = "Please enter a valid numeric PIN (digits only).";
        return;
    }

    // Call closure to get total attempt count
    let totalAttempts = getAttemptCount();

    // Perform palindrome check using Function Expression
    let isPalin = checkPalindrome(pin);

    // Get security message using Arrow Function
    let message = getSecurityMessage(isPalin);

    // BLOCK SCOPE VARIABLE inside if/else block
    if (isPalin) {
        let securityCode = "SEC-PAL-" + Math.floor(1000 + Math.random() * 9000); // Block
scoped with let
        resultDiv.className = "result success";
        resultDiv.style.display = "block";
        resultDiv.innerHTML = `
            <strong>${message}</strong><br><br>
            <strong>Entered PIN:</strong> ${pin}<br>
            <strong>Reversed PIN:</strong> ${reversePIN(pin)}<br>

```

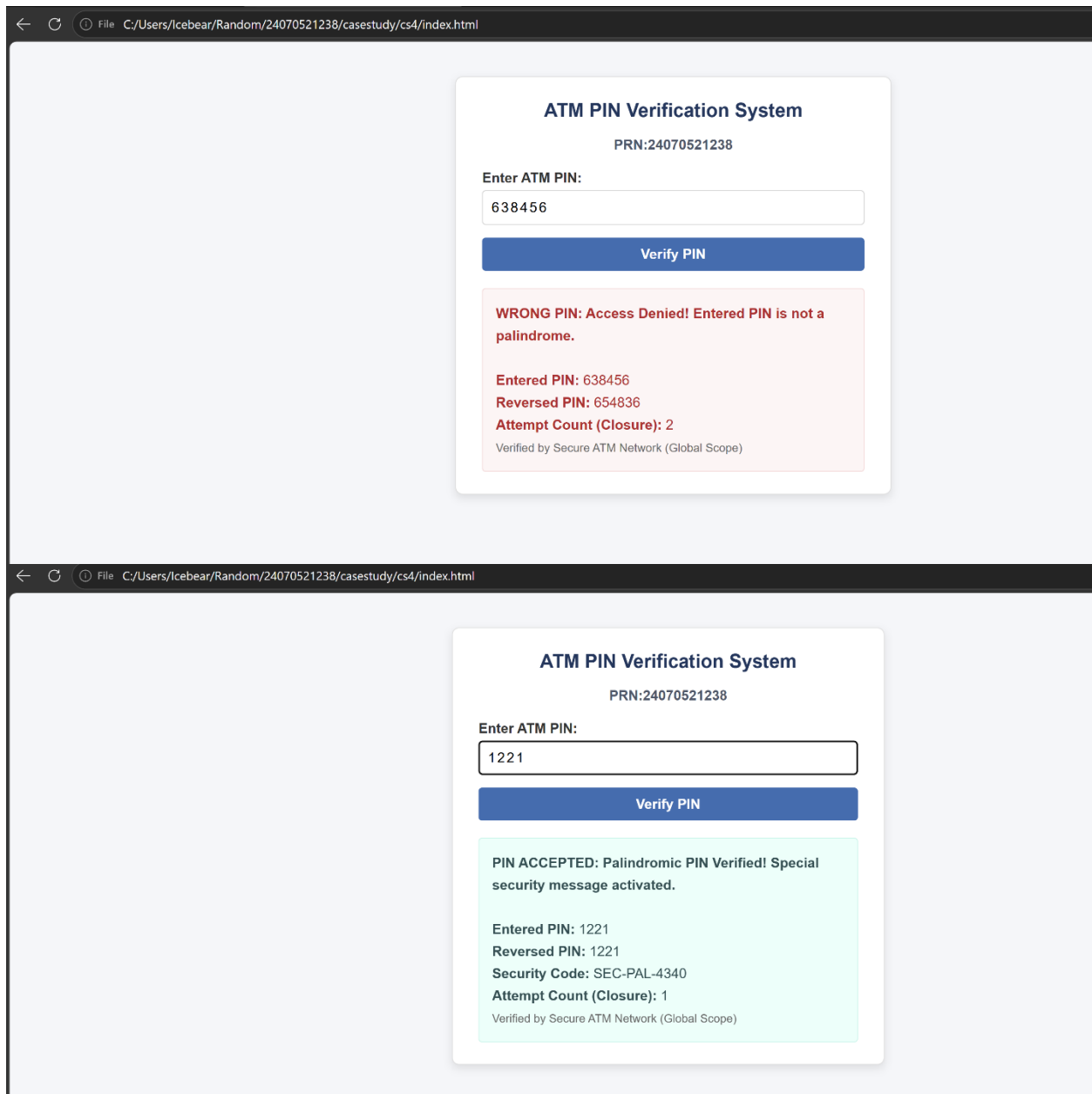
```

        <strong>Security Code:</strong> ${securityCode}<br>
        <strong>Attempt Count (Closure):</strong> ${totalAttempts}<br>
        <small style="color: #666;">Verified by ${BANK_NAME} (Global Scope)</small>
    `;
} else {
    resultDiv.className = "result error";
    resultDiv.style.display = "block";
    resultDiv.innerHTML = `
        <strong>${message}</strong><br><br>
        <strong>Entered PIN:</strong> ${pin}<br>
        <strong>Reversed PIN:</strong> ${reversePIN(pin)}<br>
        <strong>Attempt Count (Closure):</strong> ${totalAttempts}<br>
        <small style="color: #666;">Verified by ${BANK_NAME} (Global Scope)</small>
    `;
}
}
</script>

</body>
</html>

```

Output



Result / Conclusion

The ATM Card PIN Verification program was successfully implemented using JavaScript functions. The program reverses the entered PIN and checks whether it is a palindrome. **Function declaration, function expression, arrow function, scope, and closure concepts** were successfully demonstrated. The program correctly identifies whether the entered PIN reads the same forward and backward.

Experiment 5

Title: Apply Array Methods and Object Handling; Create a Cart Total Calculator with Discount Logic

Software/Tools Required

- Visual Studio Code (VS Code) or any Text Editor
- Web Browser (Google Chrome, Microsoft Edge, Mozilla Firefox, etc.)
- HTML
- JavaScript

Code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Cart Total Calculator - Exp 5</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 20px;
      max-width: 600px;
    }
    .prn {
      font-size: 16px;
      font-weight: bold;
      color: #444;
      margin-bottom: 15px;
    }
    .form-group {
      margin-bottom: 10px;
    }
    label {
      display: inline-block;
      width: 110px;
    }
    input {
      padding: 5px;
      width: 150px;
    }
    button {
      padding: 6px 12px;
      margin-top: 10px;
      cursor: pointer;
    }
    table {
      width: 100%;
      border-collapse: collapse;
      margin-top: 15px;
    }
    table, th, td {
      border: 1px solid #ccc;
      padding: 8px;
      text-align: left;
    }
    th {
      background-color: #f2f2f2;
    }
    .summary {
      margin-top: 15px;
      background: #f9f9f9;
    }
```

```

        padding: 10px;
        border: 1px solid #ddd;
    }
</style>
</head>
<body>

    <h2>Shopping Cart Calculator</h2>
    <div class="prn">PRN:24070521238</div>

    <div class="form-group">
        <label>Item Name:</label>
        <input type="text" id="itemName" placeholder="e.g. Notebook">
    </div>
    <div class="form-group">
        <label>Price (₹):</label>
        <input type="number" id="itemPrice" placeholder="e.g. 50">
    </div>
    <div class="form-group">
        <label>Quantity:</label>
        <input type="number" id="itemQty" placeholder="e.g. 2">
    </div>
    <button onclick="addItem()">Add to Cart</button>

    <br><br>
    <div class="form-group">
        <label>Discount (%):</label>
        <input type="number" id="discount" value="0" placeholder="e.g. 10">
    </div>
    <input type="button" value="Calculate Total" onclick="calculateTotal()">

    <h3>Cart Items</h3>
    <table>
        <thead>
            <tr>
                <th>Item</th>
                <th>Price</th>
                <th>Qty</th>
                <th>Total</th>
                <th>Action</th>
            </tr>
        </thead>
        <tbody id="cartTable">
            <tr>
                <td colspan="5" style="text-align:center;">Cart is empty</td>
            </tr>
        </tbody>
    </table>

    <div class="summary">
        <p>Subtotal: ₹<span id="subtotal">0.00</span></p>
        <p>Discount Amount: -₹<span id="discountAmount">0.00</span></p>
        <h4>Grand Total: ₹<span id="grandTotal">0.00</span></h4>
    </div>

    <script>
        // Array to store item objects
        let cart = [];

        // Object Constructor / Handler
        function createItemObject(name, price, qty) {
            return {
                name: name,
                price: parseFloat(price),
                quantity: parseInt(qty),
                getItemTotal: function() {
                    return this.price * this.quantity;
                }
            };
        }
    </script>

```

```

// Add Item to Cart using Array.prototype.push
function addItem() {
    let name = document.getElementById("itemName").value.trim();
    let price = document.getElementById("itemPrice").value;
    let qty = document.getElementById("itemQty").value;

    if (name === "" || price <= 0 || qty <= 0) {
        alert("Please enter valid item details.");
        return;
    }

    // Create object and push to array
    let itemObj = createItemObject(name, price, qty);
    cart.push(itemObj);

    // Clear inputs
    document.getElementById("itemName").value = "";
    document.getElementById("itemPrice").value = "";
    document.getElementById("itemQty").value = "";

    renderCart();
}

// Delete Item using Array.prototype.splice
function removeItem(index) {
    cart.splice(index, 1);
    renderCart();
}

// Render Cart using Array.prototype.forEach / Array.prototype.map
function renderCart() {
    let tbody = document.getElementById("cartTable");
    tbody.innerHTML = "";

    if (cart.length === 0) {
        tbody.innerHTML = '<tr><td colspan="5" style="text-align:center;">Cart is empty</td></tr>';
        calculateTotal();
        return;
    }

    // Object handling and Array iteration
    cart.forEach((item, index) => {
        let row = `<tr>
            <td>${item.name}</td>
            <td>${item.price.toFixed(2)}</td>
            <td>${item.quantity}</td>
            <td>${item.getItemTotal().toFixed(2)}</td>
            <td><button onclick="removeItem(${index})">Remove</button></td>
        </tr>`;
        tbody.innerHTML += row;
    });

    calculateTotal();
}

// Calculate Totals using Array.prototype.reduce
function calculateTotal() {
    // Using Array reduce method to sum item totals
    let subtotal = cart.reduce((sum, item) => sum + item.getItemTotal(), 0);

    let discountInput = parseFloat(document.getElementById("discount").value) || 0;
    if (discountInput < 0) discountInput = 0;
    if (discountInput > 100) discountInput = 100;

    let discountAmount = (subtotal * discountInput) / 100;
    let grandTotal = subtotal - discountAmount;

    document.getElementById("subtotal").innerText = subtotal.toFixed(2);
    document.getElementById("discountAmount").innerText = discountAmount.toFixed(2);
    document.getElementById("grandTotal").innerText = grandTotal.toFixed(2);
}

```

```
    }  
  </script>  
</body>  
</html>
```

Output

File C:/Users/Icebear/Random/24070521238/experiments/exp5/index.html

Shopping Cart Calculator

PRN:24070521238

Item Name:

e.g. Notebook

Price (₹):

e.g. 50

Quantity:

e.g. 2

Add to Cart

Discount (%):

10

Cart Items

Item	Price	Qty	Total	Action
Notebook	₹100.00	4	₹400.00	Remove
Book	₹1000.00	3	₹3000.00	Remove

Subtotal: ₹3400.00

Discount Amount: -₹340.00

Grand Total: ₹3060.00

Result / Conclusion

The experiment was performed successfully. Arrays and objects were used to store and manage product information, while array methods such as `reduce()` were used to calculate the total cart value efficiently. Discount logic was implemented to determine the applicable discount based on the total purchase amount, and the final payable amount was displayed correctly. This experiment demonstrated the practical use of JavaScript arrays, objects, and array methods in developing a simple shopping cart application.